

GATE Data Science & Artificial Intelligence (DA)

Question Bank: Stacks, Monotonic Patterns & Heuristic Analysis

10 Selected PYQs (GATE DA / CS) + 10 Targeted GATE-Level Practice Questions

Topic Coverage:

1. **Topic 1:** Stacks: Array/List Implementation, Call Stack Mechanics, and Standard Use Cases
2. **Topic 2:** Stack Patterns: Balanced Parentheses and Next-Greater-Element (NGE) Monotonic Variants
3. **Topic 3:** Heuristics Deep Dive: Mathematical Definitions of Admissibility and Consistency
4. **Topic 4:** Proving and Disproving Admissibility of Constructed Functions $h(n)$

Part I: 10 Previous Year Questions (PYQs)

Q1. Combination of Admissible Heuristics [GATE DA 2024 • MCQ • 1 Mark] Let h_1 and h_2 be two admissible heuristics used in A^* search for finding the shortest path to a goal. Which ONE of the following expressions is ALWAYS an admissible heuristic?

- (A) $h_1 + h_2$
- (B) $h_1 \times h_2$
- (C) $\frac{h_1 + h_2}{2}$
- (D) $|h_1 - h_2| + h_1$

Answer: (C)

Solution / Explanation:

An admissible heuristic satisfies $0 \leq h(n) \leq h^*(n)$ for all nodes n .

- Since $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$, their arithmetic mean satisfies:

$$\frac{h_1(n) + h_2(n)}{2} \leq \frac{h^*(n) + h^*(n)}{2} = h^*(n).$$

Because both are non-negative, the mean is also non-negative. Hence, $\frac{h_1 + h_2}{2}$ is always admissible.

- $h_1 + h_2$ can sum up to $2h^*(n) > h^*(n)$ when both are close to $h^*(n)$.
- $h_1 \times h_2$ can easily exceed $h^*(n)$ if heuristic values exceed 1.
- $|h_1 - h_2| + h_1$ can exceed $h^*(n)$ (e.g., if $h_1 = h^*(n)$ and $h_2 = 0$, it gives $2h^*(n)$).

Q2. Permutation Achievable via Stack [GATE CS 2023 • MCQ • 1 Mark] Consider an initially empty stack. The elements 1, 2, 3, 4 are pushed in that order. An element may be popped at any time. Which of the following output sequences CANNOT be obtained?

- (A) 2, 4, 3, 1
- (B) 1, 3, 2, 4

(C) 3, 1, 2, 4

(D) 4, 3, 2, 1

Answer: (C)

Solution / Explanation:

Let us trace each candidate sequence:

- (A) 2, 4, 3, 1: Push(1), Push(2), Pop $\rightarrow$ 2, Push(3), Push(4), Pop $\rightarrow$ 4, Pop $\rightarrow$ 3, Pop $\rightarrow$ 1. Valid.
- (B) 1, 3, 2, 4: Push(1), Pop $\rightarrow$ 1, Push(2), Push(3), Pop $\rightarrow$ 3, Pop $\rightarrow$ 2, Push(4), Pop $\rightarrow$ 4. Valid.
- (C) 3, 1, 2, 4: To output 3 first, we must push 1, 2, 3 and pop 3. The stack now contains [1, 2] with 2 on top. The next popped element **must** be 2, never 1. Thus, sequence 3, 1, 2, 4 cannot be produced.
- (D) 4, 3, 2, 1: Push all four, then pop all four. Valid.

Q3. Consistency and Triangle Inequality

[GATE CS 2020 • MCQ • 1

Mark] A heuristic function $h(n)$ is consistent (monotonic) if for every node n and every successor n' generated by action a with step cost $c(n, a, n')$:

(A) $h(n) \leq c(n, a, n') + h(n')$ and $h(\text{Goal}) = 0$

(B) $h(n) \geq c(n, a, n') + h(n')$

(C) $h(n) = c(n, a, n') - h(n')$

(D) $h(n) \leq h(n')$ for all n, n'

Answer: (A)

Solution / Explanation:

Consistency is the triangle inequality applied to heuristics: the estimated cost of reaching the goal from n cannot be greater than the step cost to reach n' plus the estimated cost from n' to the goal:

$$h(n) \leq c(n, a, n') + h(n').$$

Together with $h(\text{Goal}) = 0$, consistency guarantees that $f(n)$ never decreases along any search path and implies admissibility.

Q4. Stack Pattern: Balanced Parentheses Tracing

[GATE CS 2018 • MCQ • 1 Mark]

Consider the following string of brackets:

$$S = "[\{()\}][()]"$$

An algorithm uses a stack to determine whether S is balanced. What is the maximum number of symbols present on the stack at any one instant during processing?

(A) 2

(B) 3

(C) 4

(D) 6

Answer: (B)

Solution / Explanation:

Let us trace the stack contents as we process characters from left to right:

- Read '[': stack = ['['] (size 1)
- Read '{': stack = ['[', '{'] (size 2)
- Read '(': stack = ['[', '{', '('] (size 3) ← **Maximum reached**
- Read ')': matches '(', pop $\Rightarrow$ stack = ['[', '{'] (size 2)
- Read '}': matches '{', pop $\Rightarrow$ stack = ['['] (size 1)
- Read ']': matches '[', pop $\Rightarrow$ stack = [] (size 0)
- Read '[': stack = ['['] (size 1)
- Read '(': stack = ['[', '('] (size 2)
- Read ')': matches '(', pop $\Rightarrow$ stack = ['['] (size 1)
- Read ']': matches '[', pop $\Rightarrow$ stack = [] (size 0)

The maximum stack depth attained is 3.

Q5. Next Greater Element Time Complexity

[GATE CS 2021

• MCQ • 2 Marks] Given an array A of n distinct elements, we want to compute for each element the next greater element to its right (or -1 if none exists). Using a monotonic decreasing stack, what is the tightest worst-case time complexity of the algorithm?

- (A) $\Theta(n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n^2)$
- (D) $\Theta(\log n)$

Answer: (A)

Solution / Explanation:

In the standard monotonic stack approach, we traverse the array from left to right (or right to left):

- Each element index is pushed onto the stack exactly once.
- Each element index is popped from the stack at most once.
- Although an individual element might trigger several pops, the total number of push operations is n and the total number of pop operations across the entire algorithm is at most n .

Thus, the total number of stack operations is at most $2n$, yielding a tight worst-case time complexity of $\Theta(n)$ amortized and absolute.

Q6. Two Stacks Implementing a Queue

[GATE CS 2019 • NAT • 2

Marks] A queue is implemented using two stacks S_1 and S_2 . An enqueue operation pushes onto S_1 . A dequeue operation checks if S_2 is empty; if empty, it transfers all elements from S_1 to S_2 by popping from S_1 and pushing to S_2 , and then pops from S_2 .

Suppose 10 elements are enqueued into an initially empty queue, followed by 4 dequeue operations, then 5 more enqueue operations, and finally 3 dequeue operations. What is the total

number of push operations performed across both S_1 and S_2 ?

Answer: 25

Solution / Explanation:

Let us trace the push operations on each stack:

- First 10 enqueues: Pushed directly to S_1 .
Push count on $S_1 = 10$.
- First dequeue: S_2 is empty. All 10 elements from S_1 are popped and pushed to S_2 .
Push count on $S_2 = 10$. Then 1 popped from S_2 . (S_2 now has 9 elements).
- Next 3 dequeues: S_2 is not empty; elements are popped directly from S_2 . No pushes. (S_2 now has 6 elements).
- Next 5 enqueues: Pushed directly to S_1 .
Push count on $S_1 = 10 + 5 = 15$.
- Next 3 dequeues: S_2 currently has 6 elements, which is ≥ 3 . All 3 dequeues are served directly from S_2 without any transfer.

Total pushes on $S_1 = 10 + 5 = 15$.

Total pushes on $S_2 = 10$.

Total pushes across both stacks = $15 + 10 = 25$.

Q7. Admissibility and Scale Transformations

[GATE CS

2015 • MSQ • 2 Marks] Let $h^*(n)$ be the optimal cost from node n to a goal state, and let $h(n)$ be an admissible heuristic function such that $0 \leq h(n) \leq h^*(n)$ for all nodes n . Which of the following derived functions is/are **guaranteed** to be admissible?

- (A) $h_A(n) = \alpha \cdot h(n)$ where $0 \leq \alpha \leq 1$
- (B) $h_B(n) = h(n) + k$ for a constant integer $k \geq 1$
- (C) $h_C(n) = \lfloor h(n) \rfloor$
- (D) $h_D(n) = \max(0, h(n) - 1)$

Answer: (A, C, D)

Solution / Explanation:

- (A) is True: Since $0 \leq \alpha \leq 1$ and $h(n) \leq h^*(n)$, $\alpha h(n) \leq h(n) \leq h^*(n)$.
- (B) is False: At a goal node G , $h^*(G) = 0$. But $h_B(G) = 0 + k = k > 0 = h^*(G)$, which violates admissibility.
- (C) is True: Since $\lfloor h(n) \rfloor \leq h(n) \leq h^*(n)$ and $h(n) \geq 0$, $\lfloor h(n) \rfloor \geq 0$ is guaranteed to never overestimate.
- (D) is True: $h_D(n) \leq h(n) \leq h^*(n)$ and the $\max(0, \cdot)$ ensures non-negativity.

Q8. Monotonic Stack Output Tracing

[GATE CS 2017 • MCQ • 2 Marks]

Consider an array $A = [4, 5, 2, 25]$. What is the output of the Next Greater Element (NGE) query for each element in A from left to right?

- (A) $[5, 25, 25, -1]$
- (B) $[5, -1, 25, -1]$
- (C) $[25, 25, 25, -1]$

(D) $[5, 25, -1, -1]$

Answer: (A)

Solution / Explanation:

Let us find the first element to the right that is strictly greater for each position:

- For $A[0] = 4$: The first greater element to its right is 5.
- For $A[1] = 5$: Elements to the right are 2, 25. The first element greater than 5 is 25.
- For $A[2] = 2$: Elements to the right are 25. The first element greater than 2 is 25.
- For $A[3] = 25$: There are no elements to the right $\Rightarrow -1$.

The result array is $[5, 25, 25, -1]$.

Q9. Evaluation of Infix Expression via Two Stacks

[GATE CS 2014 • NAT • 1 Mark] An

operator-precedence parsing algorithm uses an operand stack and an operator stack to evaluate the postfix evaluation of the expression:

$$6 + 3 \times (8 - 5)$$

What integer value is popped as the final result from the operand stack?

Answer: 15

Solution / Explanation:

Following standard operator precedence:

- Parentheses evaluated first: $(8 - 5) = 3$.
- Multiplication has higher precedence than addition: $3 \times 3 = 9$.
- Finally addition: $6 + 9 = 15$.

Using operand stack $[6, 3, 8, 5]$ and operator stack $[+, \times, (, -]$:

- Pop 8, 5 and $- \Rightarrow 8 - 5 = 3$, push 3.
- Pop 3, 3 and $\times \Rightarrow 3 \times 3 = 9$, push 9.
- Pop 6, 9 and $+$ $\Rightarrow 6 + 9 = 15$, push 15.

The final returned integer is 15.

Q10. Consistency versus Admissibility Disproof

[GATE CS

2022 • MCQ • 2 Marks] Consider a graph with three nodes A, B, G , where G is the goal. Directed edges: $c(A, B) = 1$, $c(B, G) = 3$, $c(A, G) = 5$. The true optimal costs to G are $h^*(A) = 4$, $h^*(B) = 3$, $h^*(G) = 0$. A proposed heuristic assigns:

$$h(A) = 4, \quad h(B) = 1, \quad h(G) = 0$$

Which of the following describes the properties of h ?

- (A) h is admissible and consistent.
- (B) h is admissible, but NOT consistent.
- (C) h is neither admissible nor consistent.
- (D) h is consistent, but NOT admissible.

Answer: (B)

Solution / Explanation:

- **Admissibility check:** $h(A) = 4 \leq h^*(A) = 4$. $h(B) = 1 \leq h^*(B) = 3$. $h(G) = 0 \leq h^*(G) = 0$. Since $0 \leq h(n) \leq h^*(n)$ for all nodes, h is **admissible**.
- **Consistency check on edge (A, B) :** Consistency requires $h(A) \leq c(A, B) + h(B)$. Here, $h(A) = 4$, while $c(A, B) + h(B) = 1 + 1 = 2$. Since $4 \not\leq 2$, the triangle inequality is violated on edge (A, B) .

Thus, h is admissible, but **NOT** consistent.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Dynamic Array Stack Amortized Cost Analysis [GATE DA Standard • MCQ

• 1 Mark] A stack is implemented using a dynamic array with an initial capacity of 1. When the array is full, its capacity is doubled, taking $\mathcal{O}(k)$ time to copy the existing k elements to the newly allocated array. What is the amortized time complexity per push operation over a sequence of n pushes?

- (A) $\Theta(1)$
- (B) $\Theta(\log n)$
- (C) $\Theta(n)$
- (D) $\Theta(n^2)$

Answer: (A)

Solution / Explanation:

Doubling occurs at sizes $1, 2, 4, 8, \dots, 2^{\lfloor \log_2 n \rfloor}$. The total number of element copies across all reallocations is:

$$\sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j = 2^{\lfloor \log_2 n \rfloor + 1} - 1 < 2n.$$

Adding the n regular insertions, the aggregate time for n push operations is:

$$T(n) \leq n + 2n = 3n.$$

The amortized time per operation is $\frac{T(n)}{n} \leq \frac{3n}{n} = 3 = \Theta(1)$.

Practice Q2. Balanced Parentheses: Minimum Additions for Validity [GATE DA Standard • NAT • 2 Marks]

Given the bracket string:

$$S = ")) ((((("$$

What is the minimum number of parentheses that must be inserted anywhere in S to make the entire string validly balanced?

Answer: 4

Solution / Explanation:

We process the string tracking unmatched closing brackets and unmatched opening brackets:

- Character 1: ')' $\Rightarrow$ no opening bracket to pair with. Needs 1 opening bracket before it. $\text{unmatched_close} = 1$.
- Character 2: ')' $\Rightarrow$ no opening bracket. $\text{unmatched_close} = 2$.
- Character 3: '(' $\Rightarrow$ $\text{open_count} = 1$.
- Character 4: '(' $\Rightarrow$ $\text{open_count} = 2$.
- Character 5: ')' $\Rightarrow$ matches one open; $\text{open_count} = 1$.
- Character 6: '(' $\Rightarrow$ $\text{open_count} = 2$.
- Character 7: ')' $\Rightarrow$ matches one open; $\text{open_count} = 1$.
- Character 8: '(' $\Rightarrow$ $\text{open_count} = 2$.

At the end:

- We have 2 unmatched closing brackets that occurred early: requires 2 added '('.
- We have 2 unmatched opening brackets at the end: requires 2 added ')'

Total insertions required = $2 + 2 = 4$.

Practice Q3. Next Greater Element II: Circular Array Variant [GATE DA Standard • MCQ • 2 Marks] Consider a circular array $A = [3, 8, 4, 1, 2]$. In a circular array, elements wrap around (the next element after the last element is the first element). What is the Next Greater Element (NGE) for each element in A from index 0 to 4?

- (A) $[8, -1, 8, 2, 3]$
- (B) $[8, -1, -1, 2, 3]$
- (C) $[8, -1, 8, 3, 3]$
- (D) $[8, 4, 8, 2, 3]$

Answer: (A)

Solution / Explanation:

In a circular array of size $n = 5$, we can examine elements up to $2n - 1$ positions using index modulo n :

- Index 0 ($A[0] = 3$): Circular search scans $[8, 4, 1, 2]$. First greater element is 8.
- Index 1 ($A[1] = 8$): Circular search scans $[4, 1, 2, 3]$. None is strictly greater than 8 $\Rightarrow -1$.
- Index 2 ($A[2] = 4$): Circular search scans $[1, 2, 3, 8]$. First greater element is 8 (at index 1).
- Index 3 ($A[3] = 1$): Circular search scans $[2, 3, 8, 4]$. First greater element is 2 (at index 4).
- Index 4 ($A[4] = 2$): Circular search scans $[3, 8, 4, 1]$. First greater element is 3 (at index 0).

The resultant NGE array is $[8, -1, 8, 2, 3]$.

Practice Q4. Monotonic Stack: Daily Temperatures Span Count [GATE DA Standard • NAT • 2 Marks] Given the array of recorded temperatures:

$$T = [73, 74, 75, 71, 69, 72, 76, 73]$$

An algorithm uses a monotonic decreasing stack of indices to find for each day how many days one must wait until a warmer temperature occurs (or 0 if no warmer day appears). What is the value of the wait days for index 2 ($T[2] = 75$)?

Answer: 4

Solution / Explanation:

Let us trace index 2 ($T[2] = 75$):

- Index 3: $T[3] = 71 \not> 75$
- Index 4: $T[4] = 69 \not> 75$
- Index 5: $T[5] = 72 \not> 75$
- Index 6: $T[6] = 76 > 75 \implies$ Warmer day found!

The number of days to wait is:

$$\text{wait} = \text{target index} - \text{current index} = 6 - 2 = 4.$$

Practice Q5. Monotonic Stack: Sum of Subarray Minimums Pattern [GATE DA Standard • NAT • 2 Marks] For the array $A = [3, 1, 2, 4]$, we wish to find the number of contiguous subarrays in which the element 1 is the minimum element. What is this count?

Answer: 8

Solution / Explanation:

Using a monotonic stack, we determine the boundary of elements strictly greater than 1 to its left and non-decreasing to its right:

- Left boundary of 1 (at index 1): To the left is $A[0] = 3 > 1$. No other elements exist. Number of choices for the left endpoint $= 1 - (-1) = 2$ choices (start at index 0 or index 1).
- Right boundary of 1: Elements to the right are $A[2] = 2 > 1$ and $A[3] = 4 > 1$. Number of choices for the right endpoint $= 4 - 1 = 4$ choices (end at index 1, 2, 3). Wait: indices are 1, 2, 3 $\implies 4 - 1 = 3$? Wait, total indices are $\{1, 2, 3\} \implies$ length to right boundary (index 4) is $4 - 1 = 4$ choices.

Total contiguous subarrays containing index 1 where 1 is the minimum:

$$\text{Total} = (\text{left choices}) \times (\text{right choices}) = 2 \times 4 = 8.$$

Subarrays: $[1], [3, 1], [1, 2], [3, 1, 2], [1, 2, 4], [3, 1, 2, 4], [1, 2, 4], [3, 1, 2, 4]$. (8 distinct subarrays).

Practice Q6. Disproving Admissibility by Counterexample [GATE DA Standard • MCQ • 2 Marks] Let $h^*(n)$ denote the shortest distance from node n to the goal in an unweighted grid with 4-directional movement (up, down, left, right). Let (x_n, y_n) be the coordinates of node n , and (x_g, y_g) be the coordinates of the goal node. Which of the following proposed heuristics $h(n)$ is **INADMISSIBLE**?

- (A) $h_1(n) = |x_n - x_g| + |y_n - y_g|$ (Manhattan Distance)
- (B) $h_2(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$ (Euclidean Distance)
- (C) $h_3(n) = \max(|x_n - x_g|, |y_n - y_g|)$ (Chebyshev Distance)
- (D) $h_4(n) = (|x_n - x_g| + |y_n - y_g|)^2$

Answer: (D)

Solution / Explanation:

In a 4-connected grid without obstacles:

- The true shortest distance is the Manhattan distance: $h^*(n) = |x_n - x_g| + |y_n - y_g|$. With obstacles, $h^*(n) \geq |x_n - x_g| + |y_n - y_g|$.
- Manhattan distance $h_1(n)$ never exceeds $h^*(n)$, so it is admissible.
- Euclidean distance $h_2(n) \leq h_1(n) \leq h^*(n)$, so it is admissible.
- Chebyshev distance $h_3(n) \leq h_1(n) \leq h^*(n)$, so it is admissible.
- For $h_4(n)$: If $|x_n - x_g| = 2$ and $|y_n - y_g| = 0$, $h^*(n) = 2$, but $h_4(n) = 2^2 = 4 > h^*(n) = 2$. It overestimates the true cost and is therefore **inadmissible**.

Practice Q7. Consistency Verification Across Edge Transitions

[GATE DA Standard

• **MSQ • 2 Marks**] Consider two nodes u and v connected by a directed edge with cost $c(u, v) = 5$. Let $h(n)$ be a heuristic function. Which of the following assignments of $(h(u), h(v))$ **VIOLATE(S)** the consistency condition $h(u) \leq c(u, v) + h(v)$?

- (A) $h(u) = 12, h(v) = 6$
- (B) $h(u) = 9, h(v) = 4$
- (C) $h(u) = 8, h(v) = 3$
- (D) $h(u) = 10, h(v) = 4$

Answer: (A, D)

Solution / Explanation:

The consistency condition requires:

$$h(u) - h(v) \leq c(u, v) = 5.$$

Let us test each option:

- (A): $h(u) - h(v) = 12 - 6 = 6 > 5$. **Violates** consistency ($12 \not\leq 5 + 6 = 11$).
- (B): $h(u) - h(v) = 9 - 4 = 5 \leq 5$. Satisfies condition ($9 \leq 5 + 4 = 9$).
- (C): $h(u) - h(v) = 8 - 3 = 5 \leq 5$. Satisfies condition ($8 \leq 5 + 3 = 8$).
- (D): $h(u) - h(v) = 10 - 4 = 6 > 5$. **Violates** consistency ($10 \not\leq 5 + 4 = 9$).

Practice Q8. Admissibility Proof for Linear Combination

[GATE DA Standard • MCQ

• **2 Marks**] Let $h_1(n)$ and $h_2(n)$ be two distinct admissible heuristic functions. Consider the linear combination:

$$h_\lambda(n) = \lambda h_1(n) + (1 - \lambda) h_2(n)$$

For which of the following ranges of the parameter λ is $h_\lambda(n)$ **guaranteed** to be an admissible heuristic?

- (A) $\lambda \in [0, 1]$
- (B) $\lambda \in [-1, 1]$
- (C) $\lambda \in [1, 2]$
- (D) Any real $\lambda \in \mathbb{R}$

Answer: (A)

Solution / Explanation:

Let $\lambda \in [0, 1]$. Then both $\lambda \geq 0$ and $(1 - \lambda) \geq 0$. Since $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$:

$$h_\lambda(n) = \lambda h_1(n) + (1 - \lambda) h_2(n) \leq \lambda h^*(n) + (1 - \lambda) h^*(n) = (\lambda + 1 - \lambda) h^*(n) = h^*(n).$$

Furthermore, since $h_1(n) \geq 0$ and $h_2(n) \geq 0$, $h_\lambda(n) \geq 0$. If $\lambda > 1$, then $(1 - \lambda) < 0$, which could make $h_\lambda(n)$ negative or fail non-negativity. If $\lambda < 0$, $\lambda h_1(n)$ is negative. Hence, admissibility is guaranteed if and only if $\lambda \in [0, 1]$ (convex combination).

Practice Q9. Stack Reversal Minimum Auxiliary Stacks

[GATE DA Standard

• MCQ • 1 Mark] What is the minimum number of additional (auxiliary) stacks required to reverse the order of elements in a given stack S if no other data structures (such as arrays, queues, or recursive function frames) are permitted?

- (A) 1
- (B) 2
- (C) 3
- (D) It is impossible using only stacks.

Answer: (B)

Solution / Explanation:

Let the original stack be S .

- Popping elements from S and pushing them onto a single auxiliary stack A reverses the order.
- However, if we pop from A and push back into S , the order is reversed a second time, restoring the original order!
- With 2 auxiliary stacks A and B :
 1. Pop all n elements from S to A (order inverted once).
 2. Pop all elements from A to B (order inverted twice: original order in B).
 3. Pop all elements from B back to S (order inverted three times: reversed order in S).

Thus, exactly 2 additional stacks are required.

Practice Q10. Strict Dominance and Search Efficiency **[GATE DA Standard • MSQ • 2 Marks]**

Let $h_1(n)$ and $h_2(n)$ be two consistent heuristics such that for all non-goal nodes n , $h_1(n) > h_2(n)$. Let N_1 and N_2 denote the set of nodes expanded by A^* graph search using h_1 and h_2 , respectively, with identical tie-breaking rules. Which of the following statements is/are **TRUE**?

- (A) h_1 strictly dominates h_2 .
- (B) Every node expanded by A^* using h_1 is also expanded by A^* using h_2 (i.e., $N_1 \subseteq N_2$).
- (C) A^* using h_1 can expand more nodes than A^* using h_2 .
- (D) Both h_1 and h_2 are guaranteed to find an optimal solution path.

Answer: (A, B, D)

Solution / Explanation:

- (A) is True: By definition, if $h_1(n) \geq h_2(n)$ everywhere and strictly greater on at least one state, h_1 dominates h_2 .
- (B) is True: A more informed heuristic prunes a superset of the nodes pruned by a less informed one. Under consistent tie-breaking, any node satisfying $f_1(n) < C^*$ also satisfies $f_2(n) < C^*$, ensuring $N_1 \subseteq N_2$.
- (C) is False: Dominance guarantees that the number of expanded nodes $|N_1| \leq |N_2|$; it will never expand more nodes.
- (D) is True: Consistency guarantees admissibility, which in turn guarantees that A^* graph search returns the optimal path cost C^* .

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)